

SQL Window Functions

Scenario-Based Practice Assignment

Level: Revision → Intermediate **MySQL 8+**

A complete worksheet covering the window functions you have already practiced: aggregate windows, PARTITION BY, ORDER BY, running totals, RANK, DENSE_RANK and ROW_NUMBER. **No solutions are included.**

How to Practice

1. Identify what each row represents. 2. Decide whether the calculation is overall or per group. 3. Add PARTITION BY when the calculation must restart for each group. 4. Add ORDER BY inside OVER() for running/ordered calculations. 5. For ranking, decide carefully between RANK(), DENSE_RANK(), and ROW_NUMBER().

Window Functions Covered

Concept	Practice
SUM() OVER()	Overall total while keeping every row
SUM() OVER(PARTITION BY ...)	Total for each customer/driver/warehouse
AVG() OVER(...)	Average for each group
MIN()/MAX() OVER(...)	Minimum/maximum within a group
ORDER BY inside OVER()	Running / ordered calculations
RANK()	Ranking with gaps when ties occur
DENSE_RANK()	Ranking without gaps
ROW_NUMBER()	Unique sequential row number
PARTITION BY	Restart calculation for each group

Scenario 1 — E-Commerce Orders

An online store wants customer and order analytics without losing individual order rows.

Schema

Table	Column	Type
customers	customer_id	INT PRIMARY KEY
customers	customer_name	VARCHAR(50)
customers	city	VARCHAR(50)
orders	order_id	INT PRIMARY KEY
orders	customer_id	INT FOREIGN KEY
orders	order_date	DATE
orders	order_value	DECIMAL(10,2)

Populated Data

customer_id	customer_name	city
1	Rahul	Mumbai
2	Priya	Pune
3	Amit	Nagpur
4	Sneha	Mumbai
5	Karan	Pune
6	Neha	Nagpur

order_id	customer_id	order_date	order_value
101	1	2026-01-05	5000
102	2	2026-01-08	7000
103	1	2026-01-15	3000
104	3	2026-01-20	9000
105	4	2026-01-25	4000
106	2	2026-02-03	5000
107	5	2026-02-10	8000
108	3	2026-02-15	6000
109	1	2026-02-20	7000
110	6	2026-02-25	5000
111	4	2026-03-02	6000
112	5	2026-03-10	4000

Questions

Q1. Customer Total Spending

Show every order with the total amount spent by that customer.

Hint: SUM() OVER(PARTITION BY customer_id)

Q2. Customer Average Order

Show every order with that customer's average order value.

Hint: AVG() OVER(PARTITION BY customer_id)

Q3. Customer Running Spending

For each customer, calculate running spending ordered by order_date.

Hint: SUM() OVER(PARTITION BY ... ORDER BY ...)

Q4. Order Rank Per Customer

Rank each customer's orders from highest value to lowest, including ties.

Hint: RANK()

Q5. Top 2 Orders Per Customer

Find each customer's top 2 orders by value, including ties.

Hint: RANK() / DENSE_RANK()

Q6. Order Percentage of Company Revenue

For every order, calculate order_value / total company revenue × 100.

Hint: SUM() OVER()

Q7. Customer Contribution

Show each customer's total spending and percentage contribution to company revenue.

Hint: You may need an intermediate query.

Scenario 2 — Logistics & Delivery

A logistics company tracks drivers and deliveries and wants performance analytics at delivery level.

Schema

Table	Column	Type
drivers	driver_id	INT PRIMARY KEY
drivers	driver_name	VARCHAR(50)
drivers	city	VARCHAR(50)
deliveries	delivery_id	INT PRIMARY KEY
deliveries	driver_id	INT FOREIGN KEY
deliveries	delivery_date	DATE
deliveries	delivery_time_hours	DECIMAL(5,2)
deliveries	delivery_value	DECIMAL(10,2)
deliveries	status	VARCHAR(20)

Populated Data

driver_id	driver_name	city
1	Ramesh	Mumbai
2	Suresh	Pune
3	Akash	Nagpur
4	Vikram	Mumbai
5	Rohit	Pune
6	Anil	Nagpur

delivery_id	driver_id	date	time	value	status
201	1	2026-01-05	4.5	5000	Delivered
202	2	2026-01-06	6.0	7000	Delayed
203	1	2026-01-10	3.5	4000	Delivered
204	3	2026-01-12	5.0	8000	Delivered
205	4	2026-01-15	7.0	6000	Delayed
206	2	2026-01-18	4.0	5000	Delivered
207	5	2026-01-20	5.5	9000	Delivered
208	3	2026-01-22	4.0	6000	Delivered
209	1	2026-01-25	6.0	7000	Delayed
210	6	2026-01-28	3.0	5000	Delivered
211	4	2026-02-02	5.5	7000	Delivered
212	5	2026-02-05	4.5	6000	Delayed

Questions

Q8. Driver Total Delivery Value

Show every delivery with the total value delivered by its driver.

Hint: SUM() OVER(PARTITION BY driver_id)

Q9. Driver Average Delivery Time

Show every delivery with its driver's average delivery time.

Hint: AVG() OVER(PARTITION BY driver_id)

Q10. Driver Running Delivery Value

Calculate running delivery value for each driver ordered by delivery_date.

Hint: SUM() OVER(PARTITION BY ... ORDER BY ...)

Q11. Driver Performance Rank

Rank drivers based on their total delivery value, highest first.

Hint: Calculate totals first, then rank the totals.

Q12. Delivery Time Rank

Rank each driver's deliveries from fastest to slowest.

Hint: RANK() OVER(PARTITION BY driver_id ORDER BY delivery_time_hours)

Q13. Top 2 Deliveries Per Driver

Find each driver's top 2 deliveries by delivery_value, including ties.

Hint: RANK() / DENSE_RANK()

Q14. Delivery vs Driver Average

Show delivery time, driver average time, and difference = delivery time – driver average.

Hint: AVG() OVER(PARTITION BY driver_id)

Scenario 3 — Warehouse Inventory

A retailer stores products across three warehouses.

Schema

Column	Type	Meaning
inventory_id	INT PRIMARY KEY	Record ID
warehouse	VARCHAR(50)	Warehouse
product	VARCHAR(50)	Product
stock	INT	Units available
stock_value	DECIMAL(10,2)	Value of stock
last_updated	DATE	Update date

Populated Data

id	warehouse	product	stock	stock_value	date
301	Mumbai	Laptop	20	1200000	2026-01-05
302	Mumbai	Phone	50	750000	2026-01-06
303	Mumbai	Tablet	30	450000	2026-01-07
304	Pune	Laptop	15	900000	2026-01-08
305	Pune	Phone	60	900000	2026-01-09
306	Pune	Tablet	25	375000	2026-01-10
307	Nagpur	Laptop	10	600000	2026-01-11
308	Nagpur	Phone	40	600000	2026-01-12
309	Nagpur	Tablet	20	300000	2026-01-13
310	Mumbai	Monitor	25	500000	2026-01-14
311	Pune	Monitor	30	600000	2026-01-15
312	Nagpur	Monitor	15	300000	2026-01-16

Questions

Q15. Warehouse Total Inventory Value

Show each product with the total stock value of its warehouse.

Hint: SUM(stock_value) OVER(PARTITION BY warehouse)

Q16. Product Average Stock

Show each product with the average number of stock units across warehouses.

Hint: AVG(stock) OVER(PARTITION BY product)

Q17. Product Rank Within Warehouse

Rank products inside each warehouse by stock_value, highest first.

Hint: RANK()

Q18. Top 2 Products Per Warehouse

Find the top 2 products in each warehouse by stock_value, including ties.

Hint: DENSE_RANK()

Q19. Running Warehouse Inventory Value

Calculate running stock value in each warehouse ordered by last_updated.

Hint: SUM() OVER(PARTITION BY warehouse ORDER BY last_updated)

Q20. Product Contribution

Calculate stock_value / warehouse total stock value × 100 for every product.

Hint: SUM(stock_value) OVER(PARTITION BY warehouse)

Scenario 4 — Sales Performance

A company tracks salespeople across regions.

Schema + Data

sale_id	salesperson	sale_date	region	amount
401	Rahul	2026-01-05	West	10000
402	Priya	2026-01-07	South	15000
403	Amit	2026-01-10	Central	12000
404	Rahul	2026-01-15	West	8000
405	Priya	2026-01-20	South	10000
406	Amit	2026-01-22	Central	15000
407	Rahul	2026-02-05	West	12000
408	Priya	2026-02-08	South	18000
409	Amit	2026-02-12	Central	10000
410	Rahul	2026-02-20	West	15000
411	Priya	2026-02-22	South	12000
412	Amit	2026-02-25	Central	20000

Questions

Q21. Salesperson Total Sales

Show every sale with the salesperson's total sales.

Hint: SUM() OVER(PARTITION BY salesperson)

Q22. Salesperson Running Sales

Calculate each salesperson's running sales ordered by sale_date.

Hint: SUM() OVER(PARTITION BY salesperson ORDER BY sale_date)

Q23. Regional Running Sales

Calculate running sales separately for each region.

Hint: SUM() OVER(PARTITION BY region ORDER BY sale_date)

Q24. Salesperson Rank

Rank salespeople by their total sales, highest first.

Hint: Calculate totals first, then RANK().

Q25. Top Salesperson Per Region

Find the highest-performing salesperson in each region based on total sales.

Hint: Intermediate result + RANK()/ROW_NUMBER()

Final Challenge — Logistics Performance Report

Using the drivers and deliveries tables from Scenario 2, create one report containing:

- driver_name
- city
- total_delivery_value
- average_delivery_time
- fastest_delivery
- slowest_delivery
- running_delivery_value
- delivery_rank

Requirements: Use SUM() OVER(), AVG() OVER(), MIN() OVER(), MAX() OVER(), a running SUM with ORDER BY, and a ranking function. Keep individual delivery rows.

Challenge: Decide where a subquery/derived table is needed. Do not GROUP BY away the delivery-level detail.

Quick Revision

Question wording	Think of
Total for everyone	SUM() OVER()
Total for each group	SUM() OVER(PARTITION BY ...)
Average for each group	AVG() OVER(PARTITION BY ...)
Running total	SUM() OVER(PARTITION BY ... ORDER BY ...)
Highest to lowest rank	RANK()/DENSE_RANK() OVER(ORDER BY ... DESC)
Unique sequence	ROW_NUMBER()
Top N with ties	RANK()/DENSE_RANK() + filter